BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

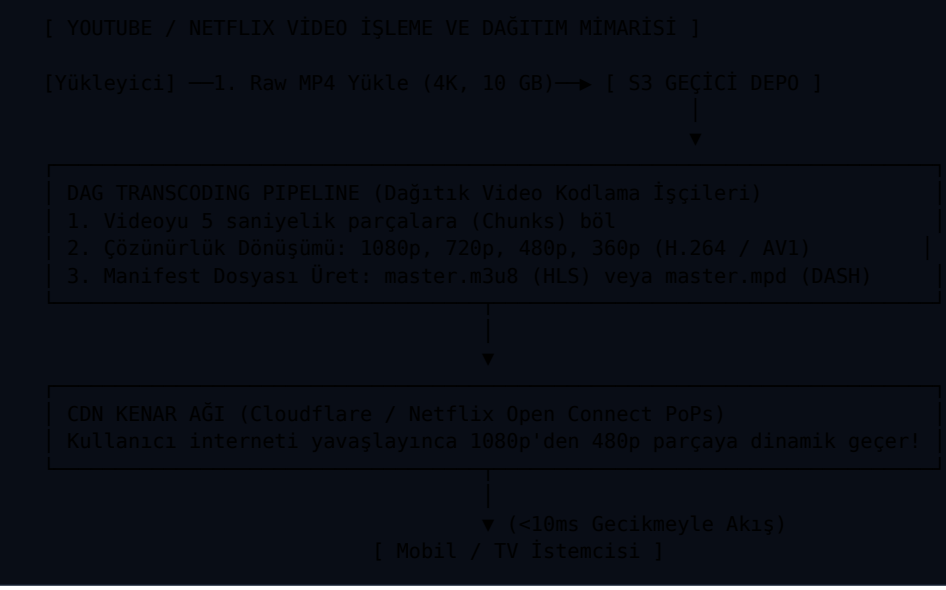
45

Video Akış Platformu (YouTube / Netflix) Tasarımı: Transcoding, Chunking ve HLS/DASH

Designing Video Streaming (YouTube / Netflix): HLS/DASH & Transcoding

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

YouTube'a 10 GB'lık dev bir film yüklediğinizde, bir kullanıcının o filmi tek parça halinde indirmesini beklemezsiniz; telefonun hafızası dolar ve interneti donardı! YouTube videoyu bir ekmek gibi 5 saniyelik dilimlere (Chunks) böler. Üstelik her dilimin 4K, 1080p, 720p ve 360p olmak üzere 4 farklı kalitede kopyasını çıkarır (Transcoding). Siz metroya binip internetiniz zayıfladığında video durmaz; oynatıcı anında 1080p dilim almayı bırakıp 360p dilimi almaya başlar (Adaptif Akış - HLS)! Metrodan çıkınca tekrar kristal netliğinde 4K dilimlere geri döner!

🇬🇧 ENGLISH (ELI5)

When uploading a 10GB 4K raw film to YouTube, you don't stream that gargantuan file in a single monolithic blob; mobile devices would freeze and crash! YouTube slices the video like a loaf of bread into hundreds of 5-second crumbs called 'Chunks'. Furthermore, an automated transcoding farm converts each crumb into 1080p, 720p, 480p, and 360p bitrates. When you step into an elevator and mobile signal weakens, playback doesn't halt; the player dynamically switches from fetching 1080p chunks to 360p chunks (Adaptive Bitrate Streaming via HLS/DASH) seamlessly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Video Platformu Mimarisi: 1) Video Yükleme ve DAG Pipeline: Video S3'e yüklendikten sonra bir Kafka olayı dağıtık kodlama motorunu (Transcoding Cluster - FFmpeg) tetikler. DAG (Directed Acyclic Graph) yapısıyla video ses, altyazı ve video parçalarına ayrıştırılır. 2) Adaptif Bit Hızı Akışı (Adaptive Bitrate Streaming - ABR): Endüstri standartları Apple HLS (`.m3u8` manifest ve `.ts` segmentleri) ve MPEG-DASH'tir (`.mpd`). Oynatıcı istemci, ağ bant genişliğini milisaniye seviyesinde ölçer ve bir sonraki 5 saniyelik parçayı en uygun çözünürlükten ister. 3) CDN Dağıtımı: Video segmentleri değiştirilemez (Immutable) statik dosyalardır; CDN PoP noktalarında %99 oranında önbelleğe alınır. Netflix'in Open Connect mimarisi, popüler filmleri gece saatlerinde doğrudan yerel telekom ISP'lerinin veri merkezlerindeki sunuculara kopyalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Video Platforms (YouTube / Netflix): 1) Video Ingestion & DAG Transcoding: Uploaded raw video drops into an ingestion S3 bucket, spawning a Directed Acyclic Graph (DAG) workflow across GPU compute worker pools. Tasks parallelize video chunking, audio extraction, watermarking, and multi-codec encoding (H.264, H.265, AV1). 2) Adaptive Bitrate Streaming (ABR): Packaged via HLS (`.m3u8` playlists and `.ts` chunk files) or MPEG-DASH (`.mpd`). Client players dynamically monitor throughput and buffer depths, swapping bitrates up or down mid-stream without frame drops. 3) Edge CDN Distribution: Chunks are immutable static files cached at edge PoPs with 98%+ cache hit rates. Netflix Open Connect appliances deploy custom storage servers directly inside thousands of residential ISP data centers globally.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: HLS Manifest ve Adaptif Segment Çözümleyici Modeli
public class HlsManifestEngine {
    public record VideoSegment(int sequenceNumber, String resolution, String chunkUrl, int
durationSecs) {}

    public static VideoSegment selectAdaptiveChunk(int currentBandwidthKbps, int chunkIndex) {
        // İstemcinin anlık internet hızına göre en iyi çözünürlüğü seç:
        String quality = switch (currentBandwidthKbps) {
            case int b when b >= 8000 -> "4K_2160p";
            case int b when b >= 3000 -> "FHD_1080p";
            case int b when b >= 1200 -> "HD_720p";
            default                   -> "SD_360p";
        };
        return new VideoSegment(chunkIndex, quality, "/video/chunk_" + chunkIndex + "_" + quality +
".ts", 5);
    }
}

void main() {
    // Metroya girildi: Hız 9000 Kbps'den 800 Kbps'e düştü!
    var highSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(9500, 42);
    var lowSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(800, 43);

    IO.println("Evdeki Parça (Wi-Fi):  " + highSpeedChunk.resolution() + " -> " +
highSpeedChunk.chunkUrl());
    IO.println("Metrodaki Parça (Edge):" + lowSpeedChunk.resolution() + " -> " +
lowSpeedChunk.chunkUrl());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Videoları tek bir sunucuda FFmpeg ile dönüştürürüm' demeyin! 2 saatlik bir 4K video tek makinede saatlerce sürer. Çözüm: Video 5 dakikalık parçalara bölünür, 50 ayrı makinede paralel kodlanır ve sonunda manifest dosyasında birleştirilir (MapReduce tarzı paralel video işleme).

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a single server running FFmpeg to transcode 4K videos is a fatal architectural flaw. Slice raw video into discrete chunks, fan out encoding across distributed GPU worker fleets in parallel, and stitch the master manifest together.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HLS ve DASH adaptif protokolleri videoyu 5 saniyelik parçalar halinde sunar.
- İstemci oynatıcı ağ hızına göre sonraki parçanın çözünürlüğünü dinamik olarak seçer.
- Video parçaları değiştirilemez statik dosyalar olduğu için CDN'lerde %98 oranında önbelleğe alınır.

🎯 Key Takeaways for the Interview

- HLS and DASH slice video into 5-second chunks for adaptive bitrate switching.
- Client players probe network throughput continuously to fetch optimal chunk resolutions.
- Immutable video segments enable near-perfect CDN edge cache hit ratios.